Pre-lab 3 and Lab 3: HTTP and Performance

Total Points: 100 (5 points for progress check in lab)

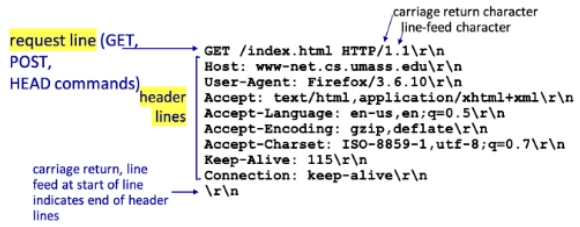
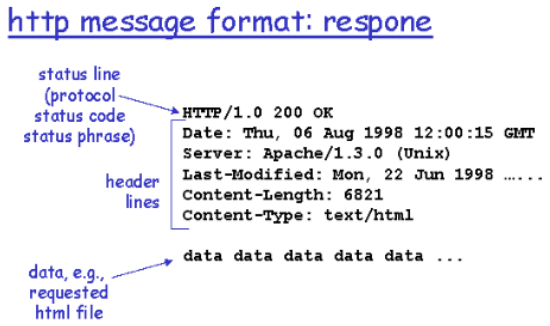
Pre-lab 3 Introduction to HTTP

Complete this pre-lab **before** attending your lab session Before beginning this lab, read Section 2.2 of the text.[1]

The HTTP protocol is the *lingua franca* of the Web. HTTP specifies methods and headers so that web clients can issue requests for web objects and receive responses from web servers. HTTP methods provide a set of actions to be performed on web resources. You will observe several of these methods in this assignment. HTTP headers are extra helpful information included in an HTTP message sent by either the client or the server. Headers are specified as name:value pairs. You can search the vast number of HTTP headers online.

Having gotten our feet wet with the Wireshark packet sniffer in Lab 2, we're now ready to use Wireshark to investigate protocols in operation. In this lab, we'll explore several aspects of the HTTP protocol: the basic request/response interaction between client and server, HTTP message formats, retrieving HTML files with multiple embedded objects, and Conditional HTTP requests. Additionally, in the last problem, we will take a look at link transmission rates and bottleneck links.

Please review class notes and the textbook to review the format of the messages exchanged. A sample client request and server response are shown below:

HTTP Request Message	HTTP Response Message
	

[5 pts] The Basic HTTP request/response messages

Match the HTTP headers to their respective definitions and usages. This exercise will help deepen your understanding of how each header contributes to the HTTP communication process, as detailed in RFC 9110

Table 1: Match the letter next to the Header with its Definition/Usage in “Answer Here”

	Header	Mat ch ← →	Answer Here	Definition/Usage
A	“**Accept-Encoding**”		**C**	Carries credentials to authenticate the requestor.
B	“**Accept**”		**I**	Contains data from the server to be stored by the client.
C	“**Authorization**”		**A**	Describes the compression formats that can be decoded.
D	“**Cache-Control**”		**B**	Details preferred content types for the response.
E	“**Connection**”		**O**	Directive to the client to store the cookie.
F	“**Content-Encoding**”		**D**	Directs how responses are to be stored by caches.
G	“**Content-Length**”		**P**	Information about the client software making the request.
H	“**Content-Type**”		**T**	Numeric indication of the outcome of the server's attempt to fulfill the request.
I	“**Cookie**”		**K**	Only send the response if the content has been modified since this date/time.
J	“**Host**”		**E**	Options for managing the current connection state.
K	“**If-Modified-Since**”		**L**	Parameters for a persistent connection.
L	“**Keep-Alive**”		**F**	Specifies the encoding used on the data.
M	“**Last-Modified**”		**R**	Text description accompanying the status code, explaining the status.
N	“**Server**”		**J**	The domain name of the server and the TCP port number.
O	“**Set-Cookie**”		**G**	The exact byte length of the HTTP body.
P	“**User-Agent**”		**H**	The media type of the body of the request/response.
Q	“**Request Version**”		**N**	The name of the server software responding to the request.
R	“**Response Phrase**”		**Q**	The protocol version used by the client in making the request.
S	“**Request Method**”		**M**	The time at which the content was last changed.
T	“**Status Code**”		**S**	The type of action the client wants the server to take on a resource.

Lab 3: HTTP and Performance

Platform: This entire lab is to be run on the Mininet VM. The lab computers are available for all of these problems.

Submission: One PDF file for this assignment is to be submitted directly on Gradescope.
Naming convention of the file: [YourCruzID].pdf.

References: Chapters 1 and 2, Computer Networking: A Top Down Approach

- Section 1.4: Delay, Loss and Throughput in Packet-Switched Networks
- Section 2.2: The Web and HTTP
- RFC 2616: Hypertext Transfer Protocol -- HTTP/1.1

Screenshots Table:
Screenshot Check-off: (just for you to keep track of your progress)

Problem #	
Q1 a,b,c	Q6 a,b
Q2 b,d,f	Q7 b,c,d,e
Q5 a,b	

Part A - Observing HTTP in Action

Q1. [25 pts] A Very Simple HTML File

Now let's explore HTTP by downloading a very simple and short HTML file with no embedded objects.
Do the following:

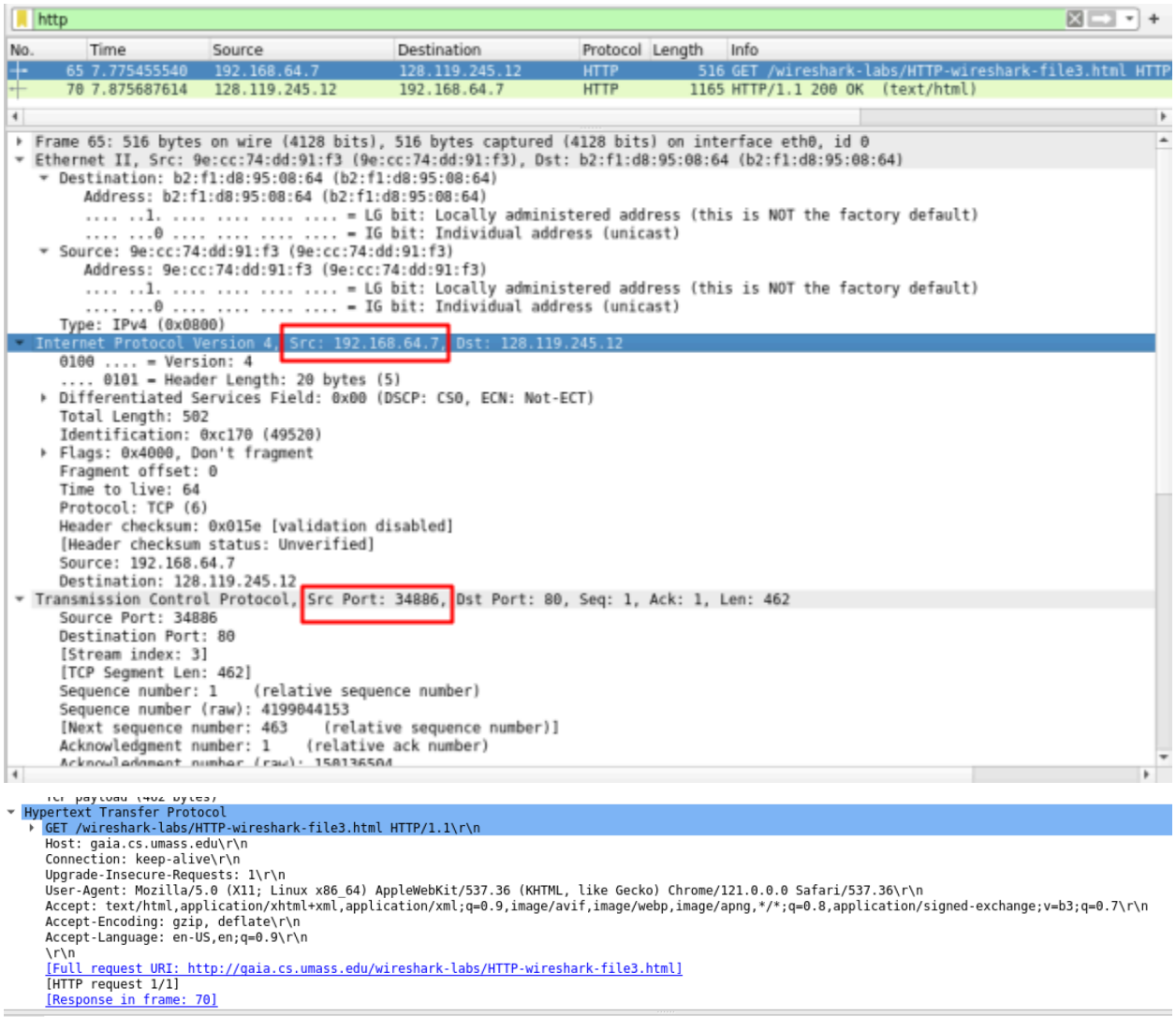
- First, open up the web browser in the Mininet VM.
- Clear the cache in the browser[2] (go to the Chromium browser setting, search Clear browsing data, select everything for all time, and clear data).
- Start up Wireshark. Set the display filter so that only HTTP messages will be displayed. (We're only interested in the HTTP protocol here and don't want to see the clutter of all captured packets).
  Then, start a packet capture using the interface **eth0**.

- Go back to the browser and enter:
  http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file3.html
  Are you familiar with the very important document displayed by your browser?!
- Stop the Wireshark packet capture, ignore the GET /favicon.ico packet.

a. **[10 pts] Table 2:** In Wireshark, examine the **Client Request Packet** to answer the questions. (All answers are visible in the packet). Include a **screenshot**, and circle the client IP Address and port number in the Wireshark output.

Table 2

Network and Transport Layer (TCP/IP)	
Questions:	**Answer** (Field Detail from Wireshark)
Example: *What is the destination port number?*	*Example Solution (from Wireshark):* *Destination port: http (80)*
What is the Client IP Address?	192.168.64.7
What is the Client port number?	34886
Application Layer (HTTP)	
Request Line (Initial Line of HTTP Request) Questions:	**Answer** (Elements of the Request Line from Wireshark)
What is the Method used for the request?	GET
What is the URI requested by the client?	/wireshark-labs/HTTP-wireshark-file3.html
What HTTP version is used by the Client?	HTTP/1.1
HTTP Headers (Following the Request Line) Questions:	**Answer** (Header Name:Value Pair from Wireshark)
What is the hostname?	gaia.cs.mass.edu
What is the Client connection management preference?	keep-alive
What is the Client browser and OS information?	Mozilla/5.0 (x11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/121.0.0.0 Safari/573.36
What types of content are accepted by the Client?	text/html, application/xhtml+xml, application/xm;q=0.9,image/avif, image/apng, */*;q=0.8, application/signed-exchange;v=b3;q=0.7
What compressed data does the Client accept?	gzip, deflate

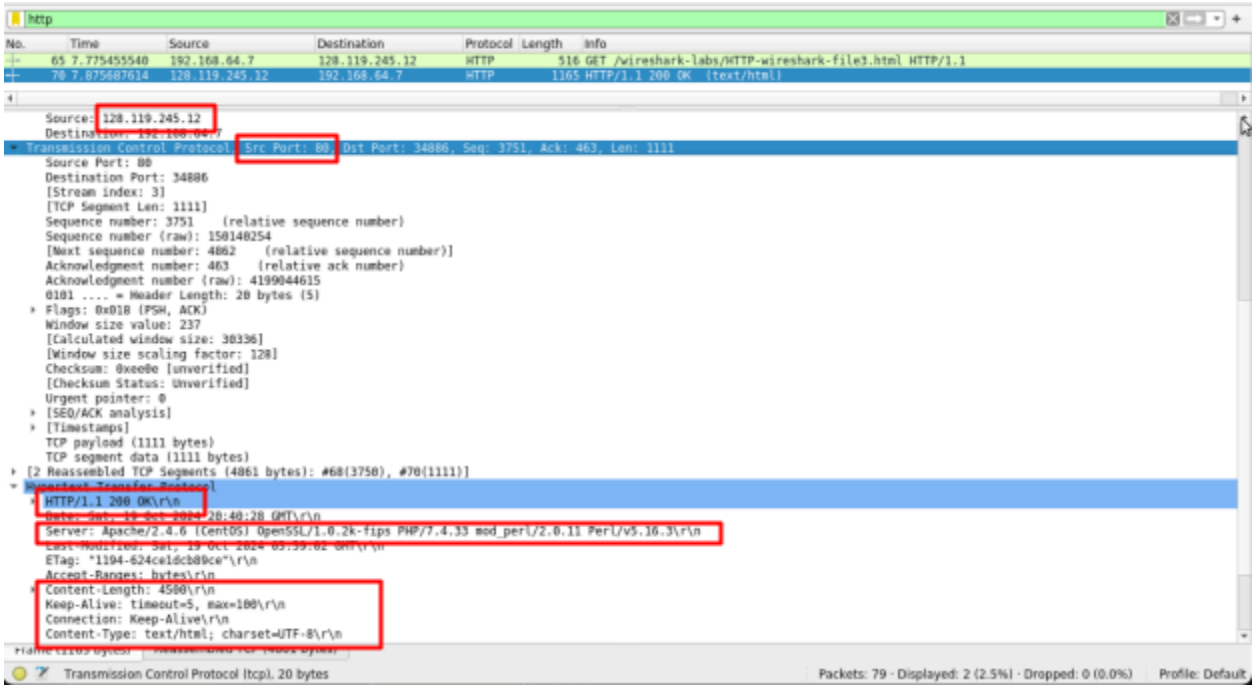


b. **[10 points] Table 3:** In Wireshark, examine the **Server Response Packet** to answer the following questions. All answers are visible in the packet. Include a **screenshot** of the packet and highlight the fields to fill in the chart.

Table 3

Network and Transport Layer (TCP/IP)	
Questions:	**Answer** (Field Detail from Wireshark)
What is the Server IP address?	128.119.245.12
What port number is the Server listening on?	80

Application Layer (HTTP)	
Status Line (Initial Line of HTTP Response) Questions:	**Answer** (Elements of the Status Line from Wireshark)
Are the Client and Server using the same version of HTTP? Explain your answer.	Yes. They are both using HTTP/1.1
What is the code that indicates the result of the Client's request?	200
What is the status phrase that accompanies the above code?	200 OK
HTTP Headers (Following the Status Line) Questions:	
What is the date and time of the server response?	Sat 19, Oct 2024 20:40:28 GMT
What is the server software information?	Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/7.4.33 mod_perl/2.0.11 Perl/v5.16.3
What is the length of the requested file?	4500 Bytes
What response does the Server provide regarding the management of the network connection?	Keep-alive: timeout=5. max=100
What type of content is being sent by the Server?	text/html; charset=UTF-8



c. **[5 points] Protocol Layers:**
For the following problem, here's a reference to HTTPS:
https://datatracker.ietf.org/doc/html/rfc2818
Now do a new Wireshark packet capture for "https://www.bankofamerica.com" without using the **http** filter. In the Wireshark captures of both "https://www.bankofamerica.com" and "http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file3.html", examine the different layers (e.g., application, transport) of network protocols.

- How do the **application and transport layers** interact **differently** in HTTP and HTTPS?
  HTTP (Hypertext Transfer Protocol) is a plaintext protocol that operates over TCP at the transport layer. In an HTTP capture, you can view the GET, POST, or other HTTP methods and see the entire content of HTTP requests and responses directly.
  HTTPS (Hypertext Transfer Protocol Secure) uses TLS/SSL (Transport Layer Security / Secure Sockets Layer) on top of HTTP. This means that the entire HTTP content is encrypted before it is sent over the transport layer (TCP). In a Wireshark capture, you won't see the content of HTTP requests and responses; instead, you'll see TLS-encrypted data between the client and server.

- If the bank transmitted a password over HTTP, would it be visible in a Wireshark capture? How does this visibility differ if transmitted when using HTTPS?
  HTTP transmits data in plaintext. If a password is transmitted over HTTP, it would be fully visible in the packet capture.

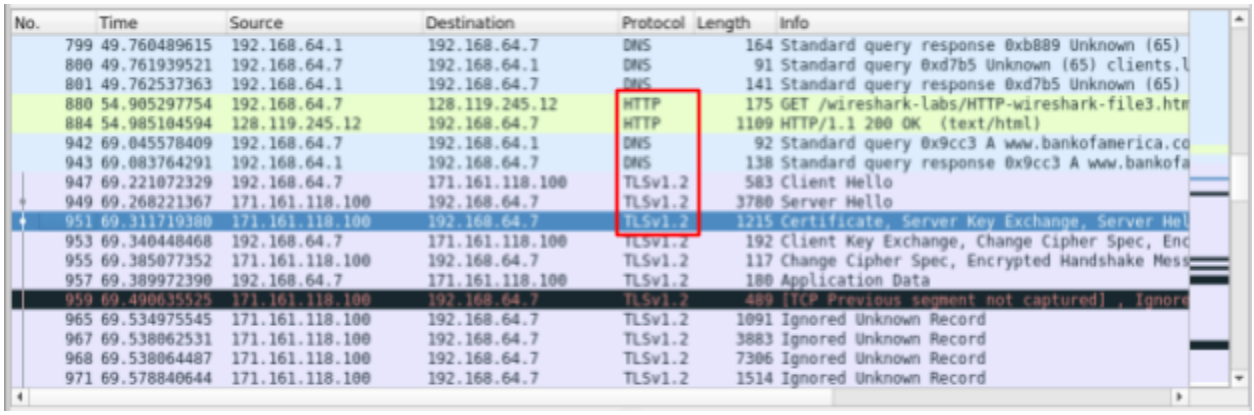
  HTTPS uses encryption via TLS/SSL. If a password or other sensitive information is transmitted over HTTPS, it would not be visible in a Wireshark capture because all HTTP data is encrypted. Instead of seeing plaintext requests and responses, you will only see encrypted data packets.

- List all the protocols that make these two captures **different**. Focus on identifying protocols that contribute to security in the HTTPS capture.
  TLS/SSL: Provides the encryption for HTTPS. In Wireshark, you may see TLS records indicating encrypted application data, handshake messages, and other security-related interactions.

  TCP: The underlying transport protocol for both HTTP and HTTPS. However, in the case of HTTPS, TCP is used to transport TLS-encrypted data.

  DNS: Both captures might show DNS requests to resolve domain names to IP addresses. Although DNS itself isn't different in HTTP vs. HTTPS, modern secure DNS protocols (like DNS over HTTPS, DoH) could be in play depending on configurations.

Include a **screenshot** and circle at least one of the protocols listed.



No.	Time	Source	Destination	Protocol	Length	Info
799	49.760489615	192.168.64.1	192.168.64.7	DNS	164	Standard query response 0xb889 Unknown (65)
800	49.761939521	192.168.64.7	192.168.64.1	DNS	91	Standard query 0xd7b5 Unknown (65) clients.l
801	49.762537363	192.168.64.1	192.168.64.7	DNS	141	Standard query response 0xd7b5 Unknown (65)
880	54.905297754	192.168.64.7	128.119.245.12	HTTP	175	GET /wireshark-labs/HTTP-wireshark-file3.htm
884	54.985104594	128.119.245.12	192.168.64.7	HTTP	1109	HTTP/1.1 200 OK (text/html)
942	69.045578409	192.168.64.7	192.168.64.1	DNS	92	Standard query 0x9cc3 A www.bankofamerica.co
943	69.083764291	192.168.64.1	192.168.64.7	DNS	138	Standard query response 0x9cc3 A www.bankofa
947	69.221072329	192.168.64.7	171.161.118.100	TLSv1.2	583	Client Hello
949	69.268221367	171.161.118.100	192.168.64.7	TLSv1.2	3780	Server Hello
951	69.311719380	171.161.118.100	192.168.64.7	TLSv1.2	1215	Certificate, Server Key Exchange, Server Hel
953	69.340448468	192.168.64.7	171.161.118.100	TLSv1.2	192	Client Key Exchange, Change Cipher Spec, Enc
955	69.385077352	171.161.118.100	192.168.64.7	TLSv1.2	117	Change Cipher Spec, Encrypted Handshake Mess
957	69.389972390	192.168.64.7	171.161.118.100	TLSv1.2	180	Application Data
959	69.490635525	171.161.118.100	192.168.64.7	TLSv1.2	489	[TCP Previous segment not captured] , Ignord
965	69.534975545	171.161.118.100	192.168.64.7	TLSv1.2	1091	Ignored Unknown Record
967	69.538062531	171.161.118.100	192.168.64.7	TLSv1.2	3883	Ignored Unknown Record
968	69.538064487	171.161.118.100	192.168.64.7	TLSv1.2	7306	Ignored Unknown Record
971	69.578840644	171.161.118.100	192.168.64.7	TLSv1.2	1514	Ignored Unknown Record

Q2. [12 pts] Networking Tools: Experiment with curl

Begin by reading up on the curl command. Determine which command option to use so that the **server returns only the HTTP headers and not the actual requested file.** Use the option to load this URL: http://www.ucsc.edu

Run Wireshark to capture the packets exchanged during curl's execution.

a. What is the full command you used?
   $ curl -I www.ucsc.edu
b. Server Response: Use the curl terminal output to answer the following questions and include a **single screenshot** with the answers to each question circled:
   - Does the server have the requested file? How do you know?
     No, the server does not have the requested file at the specified URL. This is indicated by the HTTP/1.1 301 Moved Permanently status code. It means that the requested URL (http://www.ucsc.edu) has been permanently moved to a new location (https://www.ucsc.edu/), as specified in the Location header.

   - How long has this response been cached? How is this information used?
     The Cache-Control header specifies max-age=86400, which indicates that the response can be cached for up to 86,400 seconds (24 hours).

     The Age header also shows 11067, indicating that this cached version has been stored for 11,067 seconds.

     This information helps browsers and intermediate caches know how long the response remains fresh and when they should request a new version from the origin server.

   - What type of *HTTP connection* is used? Explain what it means.
     The Connection header is set to keep-alive, which means that a persistent HTTP connection is used.

     This allows the TCP connection to stay open, enabling multiple HTTP requests and responses to be sent over a single connection without needing to establish new connections repeatedly. This reduces latency and overhead.

   - What is the size (in bytes) of the base HTML page?
     The Content-Length header indicates a size of 162 bytes for the base HTML page. This represents the size of the content being delivered in the response body.

```
mininet@mininet-vm:~/Downloads$ curl -I www.ucsc.edu
HTTP/1.1 301 Moved Permanently
Connection: keep-alive
Content-Length: 162
Content-Type: text/html
Location: https://www.ucsc.edu/
Server: nginx
X-Pantheon-Styx-Hostname: styx-fe4-a-746c76f549-bqp5h
X-Styx-Req-Id: 932e960d-8e3c-11ef-ae5e-5e7841030b85
Cache-Control: public, max-age=86400
Date: Sat, 19 Oct 2024 20:11:36 GMT
X-Served-By: cache-chi-kigq8000057-CHI, cache-sjc10035-SJC
X-Cache: HIT, HIT
X-Cache-Hits: 1, 1
X-Timer: S1729368696.462022,VS0,VE4
Vary: Cookie, Cookie
Age: 11067
Accept-Ranges: bytes
Via: 1.1 varnish, 1.1 varnish
```

<u>Now look at your Wireshark capture:</u>
Apply a filter in Wireshark and find the HTTP packet from your client to the web server. Right-click on that packet and select **"Follow TCP Stream".** You should be able to see all the packets transferred on that TCP connection.

c. Client Request:
   - According to Wireshark, which HTTP method is used for the client request? Explain the purpose of the HTTP method you observe.
     GET
     The purpose of the GET method is to request data from a specified resource on the server. It is typically used to retrieve the content of a web page or other resources without making any modifications to the server. In this case, the client is requesting the page at www.ucsc.edu using a GET request.

d. Server:
   - What is the status code of the response message from the server? What does it mean?
     The status code is 301 Moved Permanently. This indicates that the requested resource (in this case, the web page at www.ucsc.edu) has been permanently moved to a new URL, which is provided in the Location

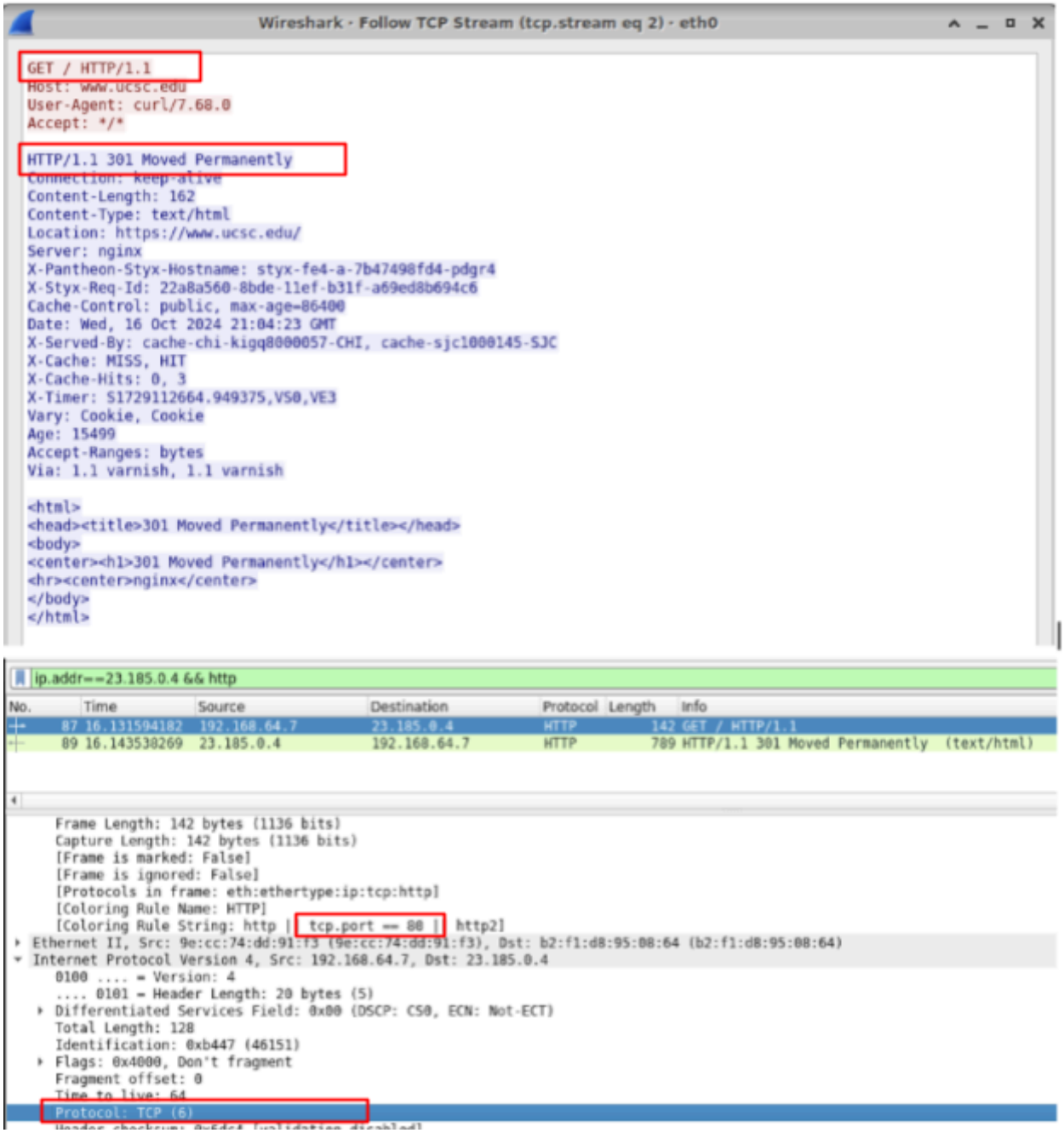
header (https://www.ucsc.edu/). This status code instructs the client to make future requests to the new URL.

- On what port number is the server listening for HTTP requests? Explain why this port is being used.
  Port number: 80

  The server is listening on port 80 for HTTP requests. Port 80 is the default port for HTTP traffic, which is why it is used here. It's a well-known port number that clients automatically use when making unencrypted HTTP requests unless a different port is specified.

- Which Transport layer protocol is used for these requests?
  TCP

Include a Wireshark **screenshot** with the responses given above highlighted.

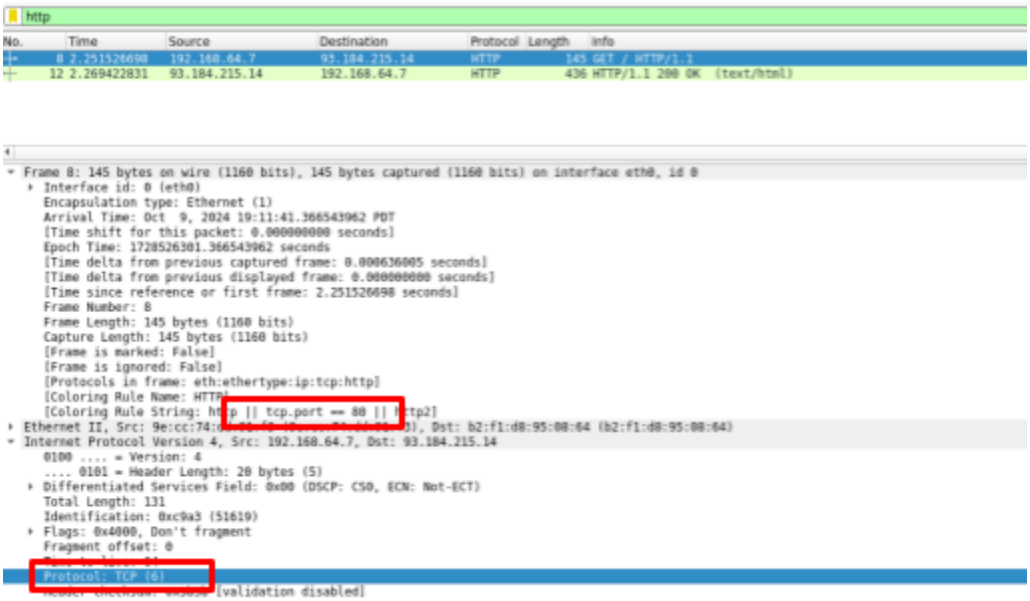


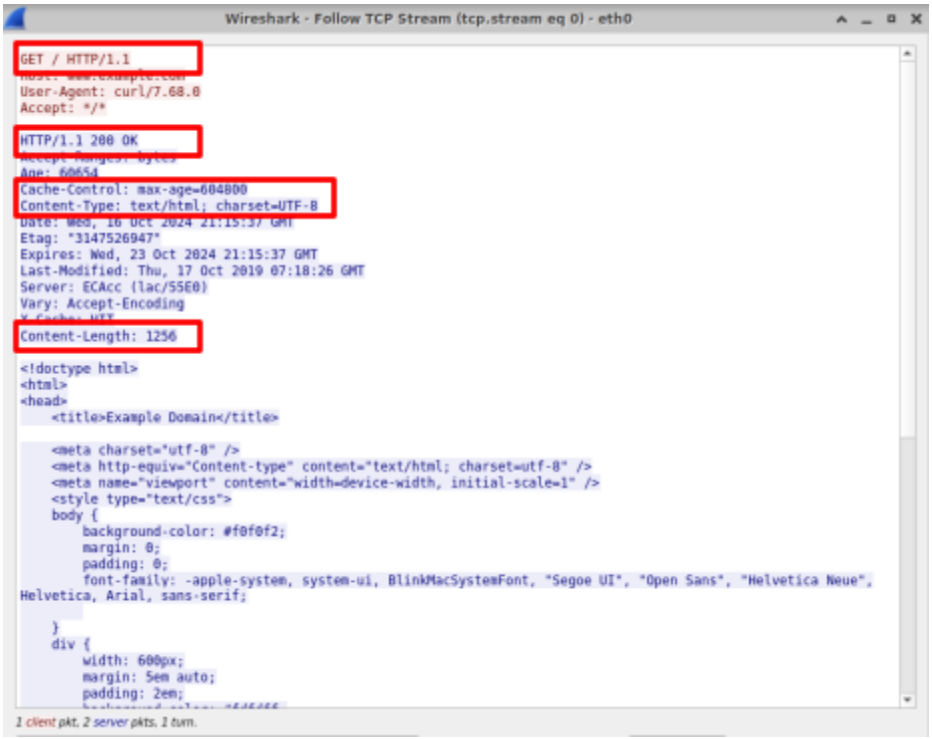
e. Explain the concept of a TCP stream in the context of loading a webpage.
TCP stream follows the relative stream related to the HTTP connection.

A TCP stream refers to the sequence of data exchanged between a client and server over a single TCP connection. When loading a webpage, the TCP stream includes all HTTP requests sent from the client to the server and the corresponding HTTP responses sent back from the server to the client. Each

TCP stream is uniquely identified by the combination of the source IP address, destination IP address, source port, and destination port.

f. Now use curl to load the response headers of http://www.example.com/
Describe the response displayed in the output terminal.
Include a **screenshot** of the Follow TCP Stream with circles around fields that were used in the above questions/answers.





Q3. [16 pts] Looking at Embedded Objects

Let's look at what happens when your browser downloads a file with references to several embedded objects, i.e., a file that includes references to other objects, accessible by hyperlink, that are stored on a server(s).

Do the following:

- Enter http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file4.html in your browser to display a short HTML file with two embedded images.
- Start a Wireshark capture.
- Note that two images are referenced in the base HTML file by URL only – they are not contained in the HTML file. As discussed in the textbook (and class), your browser must retrieve these objects from the indicated websites.
- Stop the Wireshark packet capture and filter for **http** packets.
- ***Ignore the /favicon.ico request/response in Wireshark***.

a. [10 points] Table 4:

Find the response packet associated with the first GET request in Wireshark:	
Questions:	**Answers:**
Describe the purpose of the very first GET request and the corresponding response message.	The purpose of the very first GET request is to retrieve the base HTML file from the server. This HTML file contains the content and structure of the webpage as well as references (via <img> tags) to the two images that are embedded.
Was it successful? Explain how you know.	Yes it was successful because 200 OK status code was received
If successful, how many bytes of the requested file does the Server send?	942 bytes
Copy the lines from the base **HTML** page that reference the two images (Locate the raw data in this packet and look for the <img> tags).	<img src="http://gaia.cs.umass.edu/pearson.png" WIDTH="140" HEIGHT="82"> <img src="http://kurose.cslash.net//8E_cover_small.jpb width="168" height="220">
Enter each URL separately into your browser, what do you see?	I can see the individual image files displayed
From the Wireshark trace, how many HTTP GET request messages did your browser send in total? ***(Ignore /favicon.ico in Wireshark***)	Ignoring the favicon.ico, there were 3 GET messages
Embedded images: Using Wireshark, identify the two HTTP client requests for the embedded images and their corresponding server responses.	

One image is hosted on the same server as the main HTML file. How can you prove which image is hosted on a **different** server?	To prove which image is hosted on a different server, compare the IP addresses in the Server IP Address field for each image request. The image hosted on the same server should share the same IP address as the main HTML file. An image hosted on a different server will have a different IP address in its response.
Which image is hosted on a **different** server?	The 8E_cover_small.jpg is hosted on a different server because the request is sent to 178.79.137.164 which is different from the main HTML's IP address (The pearson.png has the same IP address as the server,

b. **[6 points]** Draw a timing diagram to show the sequence of interactions between the client and server when loading the website in (a).

Your diagram should include:

- The client (your web browser), one server hosting the HTML file, and the other server that hosts one of the embedded images.
- The initial GET request for the HTML file and the server's response.
- Subsequent GET requests for each embedded image and the server response.
- Use arrows to indicate the direction of each request and response.
- There should be a clear order of interactions, labeling each step to show the progression from the initial request to the complete rendering of the web page with all embedded images.

Image 1 is the pearson image, and image 2 is the 8E_cover_small.jpg

```
Client (Browser)                Server A (HTML)                       Server B (Image)
        |                               |                                     |
        | 1. GET HTML file              |                                     |
        |------------------------------>|                                     |
        |                               |                                     |
        |          2. Response (HTML)   |                                     |
        |<------------------------------|                                     |
        |                               |                                     |
        |                               |                                     |
        | 3. GET image1 (Server A)      |                                     |
        |------------------------------>|                                     |
        |                               |                                     |
        |          4. Response (Image1) |                                     |
        |<------------------------------|                                     |
        |                               |                                     |
        |                               |                                     |
        | 5. GET image2 (Server B)      |                                     |
        |-------------------------------|------------------------------------>|
        |                               |                                     |
        |                               |        6. Response (Image2)         |
        |<----------------------------------------------------------------------|
        |                               |                                     |
```

Q4. [5 pts] Extra Credit - Optional HTTP Conditional GET

Recall from Section 2.2.5 that most web browsers perform object caching and thus often perform a conditional GET when retrieving an HTTP object. Before performing the steps below, make sure your browser's cache is empty[2].

Now do the following (follow instructions closely):

- Start up your web browser, and make sure your browser's cache is cleared.
- Start up Wireshark and listen on the **eth0** interface.

1st HTTP GET:

Enter this URL into your browser:
http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html
Your browser should display a very simple five-line HTML file.

2nd HTTP GET:

Quickly enter the same URL into your browser again (or simply select the refresh button on your browser)

- Stop the Wireshark packet capture and filter to display **HTTP** packets.

(Ignore the /favicon.ico request/response in Wireshark.)

Questions:

a. Why might your browser issue a Conditional GET?
It may be issued to check if the cached version of a resource is still up to date

1st HTTP GET and server response:

b. Is the first HTTP GET request from your browser to the server a Conditional GET? Explain your answer referencing the Wireshark trace.
No. The wireshark header does not contain headers like If-modified since the browser is yet to have a cached version of the file.

2nd HTTP GET and server response:

c. Now inspect the contents of the second HTTP GET request from your browser to the server.
   - Describe how the second HTTP GET message is different from the first one.
   - What is the "IF-MODIFIED-SINCE:" header and its value, and why is it used?
   - What is the date and its value, what does it mean?

It is sending a conditional get.
If-Modified-Since: Sat, 21 Oct 2024 07:28:00 GMT
It means that if the content was modified since the time 'Sat, 21 Oct 2024 07:28:00 GMT,' the cached content is no longer valid and a new version should be requested from the server.

Part B - Measuring Performance in Mininet

These problems will explore network performance using your Mininet Topology from Lab2. The **Default Configuration** is defined as having all link rates set to 20 ms in the topology script.

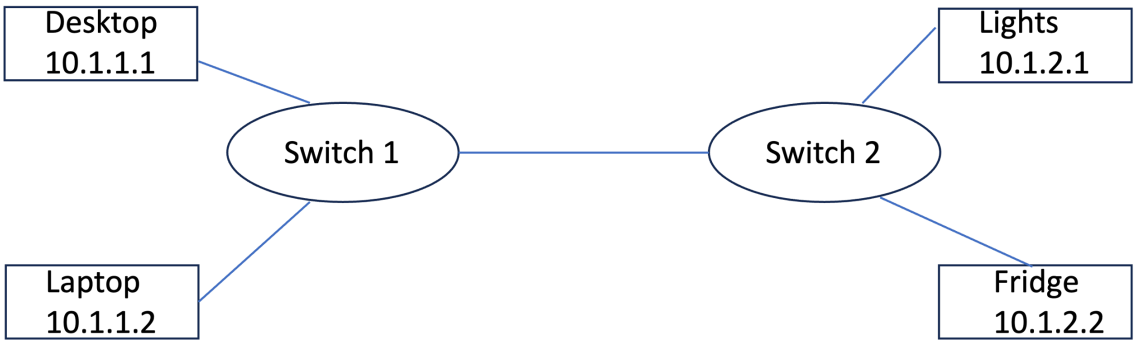


Figure 1

Propagation Delay

Q5. [6 pts] Exploring Propagation Delay 1

Link delay can be changed using: `self.addLink(Switch1, Switch2, delay="10ms")`

.

a. Edit your topology script from Lab2 and set all link delays to the Default Configuration – Run the following tests:
   - `Desktop ping -c 3 Laptop`
   - `Laptop ping -c 3 Lights`

b. Increase Link Delay – Set all links to have a delay of 40 ms and run the same 2 tests as above.
   <u>Attach both screenshots(5a and 5b)</u> of your results, highlight the changes, if any, and <u>explain</u> your observations with respect to the result:. Do you notice any changes in End-to-End delay? Is this what you expected? Why or why not?
   Results from 5a:

```
mininet@mininet-vm:~/Downloads$ sudo python3 ypark55-topo.py
mininet> Desktop ping -c 3 Laptop
PING 10.1.1.2 (10.1.1.2) 56(84) bytes of data.
64 bytes from 10.1.1.2: icmp_seq=1 ttl=64 time=220 ms
64 bytes from 10.1.1.2: icmp_seq=2 ttl=64 time=98.4 ms
64 bytes from 10.1.1.2: icmp_seq=3 ttl=64 time=94.2 ms

--- 10.1.1.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2015ms
rtt min/avg/max/mdev = 94.172/137.621/220.245/58.449 ms
mininet> Laptop ping -c 3 Lights
PING 10.1.2.1 (10.1.2.1) 56(84) bytes of data.
64 bytes from 10.1.2.1: icmp_seq=1 ttl=64 time=280 ms
64 bytes from 10.1.2.1: icmp_seq=2 ttl=64 time=147 ms
64 bytes from 10.1.2.1: icmp_seq=3 ttl=64 time=134 ms

--- 10.1.2.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2013ms
rtt min/avg/max/mdev = 134.480/187.131/280.403/66.135 ms
```

Results from 5b:

```
mininet@mininet-vm:~/Downloads$ sudo python3 ypark55-topo.py
mininet> Desktop ping  -c 3 Laptop
PING 10.1.1.2 (10.1.1.2) 56(84) bytes of data.
64 bytes from 10.1.1.2: icmp_seq=1 ttl=64 time=349 ms
64 bytes from 10.1.1.2: icmp_seq=2 ttl=64 time=177 ms
64 bytes from 10.1.1.2: icmp_seq=3 ttl=64 time=164 ms

--- 10.1.1.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 163.689/230.030/349.490/84.643 ms
mininet> Laptop ping -c 3 Lights
PING 10.1.2.1 (10.1.2.1) 56(84) bytes of data.
64 bytes from 10.1.2.1: icmp_seq=1 ttl=64 time=514 ms
64 bytes from 10.1.2.1: icmp_seq=2 ttl=64 time=266 ms
64 bytes from 10.1.2.1: icmp_seq=3 ttl=64 time=248 ms

--- 10.1.2.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 247.660/342.577/514.246/121.614 ms
```

The end-to-end delay increased noticeably (highlighted in yellow), nearly doubling for all packets. This was expected because the delay between the links has been increased from 20ms to 40ms. Since the propagation delay between each node increases, it is natural that the end-to-end delay increases.

Q6: [6 pts] **Changing INDIVIDUAL link delay**

Let's pretend we have a trans-oceanic link:

a. Reset your network to the Default configuration, but set the link between Switch 1 and Switch 2 to 500 ms. What do you observe when you ping from the Laptop to the Fridge?

```
mininet> Laptop ping -c 3 Fridge
PING 10.1.2.2 (10.1.2.2) 56(84) bytes of data.
64 bytes from 10.1.2.2: icmp_seq=1 ttl=64 time=2421 ms
64 bytes from 10.1.2.2: icmp_seq=2 ttl=64 time=1378 ms
64 bytes from 10.1.2.2: icmp_seq=3 ttl=64 time=1194 ms

--- 10.1.2.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2070ms
rtt min/avg/max/mdev = 1193.811/1664.257/2421.026/540.370 ms, pipe 3
```

There is a significant increase in the RTT. Even the lowest RTT is 1194 ms. This is due to the increased delay between switch 1 and switch 2.

b. Now ping between the Laptop and the Desktop. Are the results expected?

```
mininet> Laptop ping -c 3 Desktop
PING 10.1.1.1 (10.1.1.1) 56(84) bytes of data.
64 bytes from 10.1.1.1: icmp_seq=1 ttl=64 time=346 ms
64 bytes from 10.1.1.1: icmp_seq=2 ttl=64 time=178 ms
64 bytes from 10.1.1.1: icmp_seq=3 ttl=64 time=168 ms

--- 10.1.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2009ms
rtt min/avg/max/mdev = 168.495/230.593/345.736/81.502 ms
```

The results are expected. The delay from switch 1 to switch 2 was increased to 500ms. However, the Laptop and Desktop nodes do not have to go across the switch1-to-switch2 link in order to transfer packets. Hence, there is no significant increase in its RTT.

c. Discuss: Considering End-to-End link delay, how does the delay between Switch 1 and Switch 2 impact the ping between Laptop-Fridge compared to Laptop-Desktop? If there is a difference, what is the cause of the difference?

The delay between switch 1 and switch 2 has a significant impact on the ping between Laptop-Fridge, but none on the Laptop-Desktop. This is due to the fact that Laptop-Fridge ping needs to go through the switch1-switch2 link, which has a substantial delay. On the other hand, Laptop-Desktop does not have to go through the switch1-switch2 link, and hence shows less delay that that of the Laptop-Fridge ping.

Include a **screenshot** of the two Ping runs for parts A and B and use the results and the topology as the basis of your discussion.

Throughput

In this section, we continue our study of network performance and focus on throughput. We make a distinction between transmission rate (often observed at the sender) and throughput. Throughput is the rate data is delivered to the receiver and is specifically observed at the Application Layer.

Note: Remember that rates are measured in bits per second and when the numbers are too large we use notation such as Gpbs (gigabits per second) and Mbps (megabits per second). Typically these rates are stated in bits (not bytes). G = 10^9 and M = 10^6

Q7. [25 pts] Exploring Transmission Rate and Throughput

The bandwidth of the links can be altered by adding a `bw = ` parameter to addLink (which takes its argument as a number **representing the bandwidth in Mbps**).

a. Research the **iperf command** and describe it in your own words.
iperf is a network performance measurement tool used to test the bandwidth between two hosts on a network. It can measure the maximum TCP or UDP throughput by creating a data flow between a client and a server and reporting the transfer rates and other statistics.

b. Understanding iperf: Set the bandwidth of all links to **1 Gbps**.

Run the `iperf` command between the Desktop and Fridge in the Mininet command line and **screenshot** the output. Explain the iperf output – What is the meaning of the rates displayed and are they what you expected?

```
mininet> iperf Desktop Fridge
*** Iperf: testing TCP bandwidth between Desktop and Fridge
*** Results: ['951 Mbits/sec', '953 Mbits/sec']
```

The rates displayed indicate the TCP bandwidth (aka. The amount of data that was successfully transferred between the two hosts (Desktop & Fridge))

The result was as expected. The bandwidth was configured to 1Gbps for all links, and so the expected result was a throughput close to 1Gbps. The results 951 to 953 Mbps throughput are close to 1gbps. (I had decreased all the delay between the nodes to 0ms to decrease overhead)

c. First and Last Hop: Change the bandwidth of edge (host-to-switch) links to **200 Mbps**, keeping the bandwidth of the link switch1-switch2 = 1Gbps. Run **iperf** 3 times between the Desktop and Fridge and find the average throughput seen by the Fridge. Discuss: Are your results what you expected? Which links have the greatest influence over the throughput observed? Explain carefully.

Include a **screenshot** of the results.

```
mininet> iperf Desktop Fridge
*** Iperf: testing TCP bandwidth between Desktop and Fridge
*** Results: ['182 Mbits/sec', '183 Mbits/sec']
mininet> iperf Desktop Fridge
*** Iperf: testing TCP bandwidth between Desktop and Fridge
*** Results: ['183 Mbits/sec', '184 Mbits/sec']
mininet> iperf Desktop Fridge
*** Iperf: testing TCP bandwidth between Desktop and Fridge
*** Results: ['182 Mbits/sec', '183 Mbits/sec']
mininet>
```

The average throughput was 182.83 Mbits/sec
The results were as expected based on the configuration. The bandwidth of the edge links was set to 200Mbps. Even though the link between switch1 and switch2 is 1 Gbps, the throughput observed is constrained by the slower edge links.

The links with the greatest influence are Desktop –Switch 1 and Switch 2 – Fridge, which serve as a bottleneck. Even though the link between the switches is 1 Gbps, the overall throughput is determined by the slowest part of the network path, which in this case are the 200 Mbps edge links, and specifically the edge links of Desktop –Switch 1 and Switch 2 – Fridge.

d. Sharing the network: **Make sure to keep the same setting as part C.** We will now generate simultaneous traffic between Desktop-Fridge and Desktop-Lights. For this, we can use xterm :

xterm Desktop Fridge Lights

Configure "Desktop" as an iperf server using `iperf -s`.

iperf -s <assign a server port>

Then generate iperf traffic from Lights and Fridge for 5 seconds as clients using:

```
iperf -c <server IP> -p <server port> -t <seconds>
```

Be sure to run both `iperf -c` commands simultaneously. Repeat 3 times and average the results.

Discussion: Which links were shared by the two traffic flows, and what are their bandwidths? Explain your initial expectation with respect to your actual observations. Highlight results in your screenshot to support your discussion.

Include a **screenshot** of the results (this should have 4 terminal windows, 1 mininet, and 3 Xterm - Server, Desktop, SmartTv).

The links shared by the two traffic flows are the link from switch1–switch2 and Desktop–switch1.

The bandwidth of the two switches1 to 2 is 1gbps and 200 mbps for desktop-switch1.

I would expect the combined throughput of the two traffic flows would be around 200mbps because the switch1–Desktop link is a bottleneck. (it could be over 200 mbps because the two traffic flows do not begin at the exact same time)

Actual observations

The actual observations fit my expectations because the sum of each of the workflows were close or equal to 200ms.

Desktop ↔ Fridge results in bandwidths around:

- 67.2 Mbits/sec in one instance
- 95.0 Mbits/sec in another instance
- 106 Mbits/sec in the final instance

Desktop ↔ Lights results in bandwidths around:

- 125 Mbits/sec in one instance
- 95.5 Mbits/sec in another instance
- 104 Mbits/sec in the final instance

Mininet:

```
mininet@mininet-vm:~/Downloads$ sudo python3 ypark55-topo.py
mininet> iperf Desktop Fridge
*** Iperf: testing TCP bandwidth between Desktop and Fridge
*** Results: ['182 Mbits/sec', '183 Mbits/sec']
mininet> iperf Desktop Fridge
*** Iperf: testing TCP bandwidth between Desktop and Fridge
*** Results: ['183 Mbits/sec', '184 Mbits/sec']
mininet> iperf Desktop Fridge
*** Iperf: testing TCP bandwidth between Desktop and Fridge
*** Results: ['182 Mbits/sec', '183 Mbits/sec']
mininet> xterm Desktop Fridge Lights
mininet>
```

Xterm:

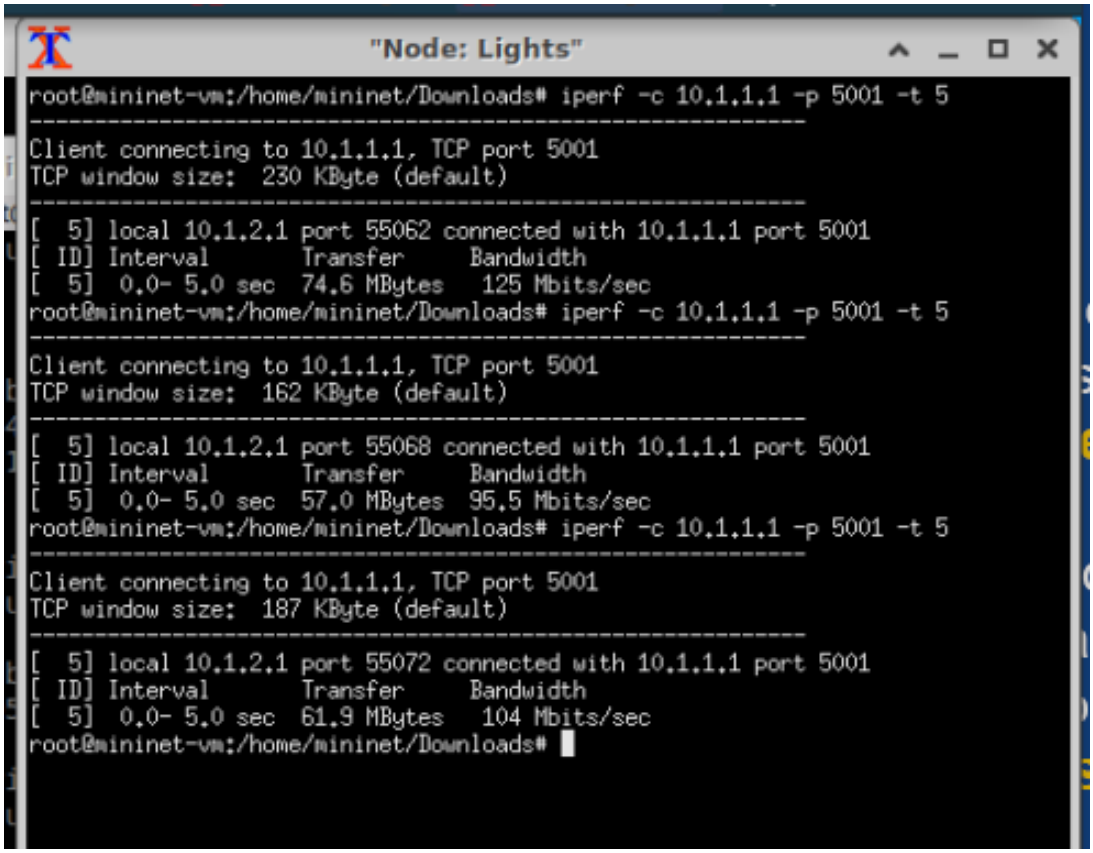
Desktop

```
"Node: Desktop"
root@mininet-vm:/home/mininet/Downloads# iperf -s 5001
iperf: ignoring extra argument -- 5001
------------------------------------------------------------
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[  6] local 10.1.1.1 port 5001 connected with 10.1.2.1 port 55062
[  7] local 10.1.1.1 port 5001 connected with 10.1.2.2 port 56516
[ ID] Interval       Transfer     Bandwidth
[  6]  0.0- 5.1 sec  74.6 MBytes   123 Mbits/sec
[  7]  0.0- 5.0 sec  40.1 MBytes  66.8 Mbits/sec
[  6] local 10.1.1.1 port 5001 connected with 10.1.2.2 port 56518
[  7] local 10.1.1.1 port 5001 connected with 10.1.2.1 port 55068
[  6]  0.0- 5.1 sec  63.4 MBytes   104 Mbits/sec
[  7]  0.0- 5.0 sec  57.0 MBytes  95.0 Mbits/sec
[  6] local 10.1.1.1 port 5001 connected with 10.1.2.2 port 56522
[  7] local 10.1.1.1 port 5001 connected with 10.1.2.1 port 55072
[  6]  0.0- 5.1 sec  64.6 MBytes   107 Mbits/sec
[  7]  0.0- 5.0 sec  61.9 MBytes   103 Mbits/sec
```

Fridge:

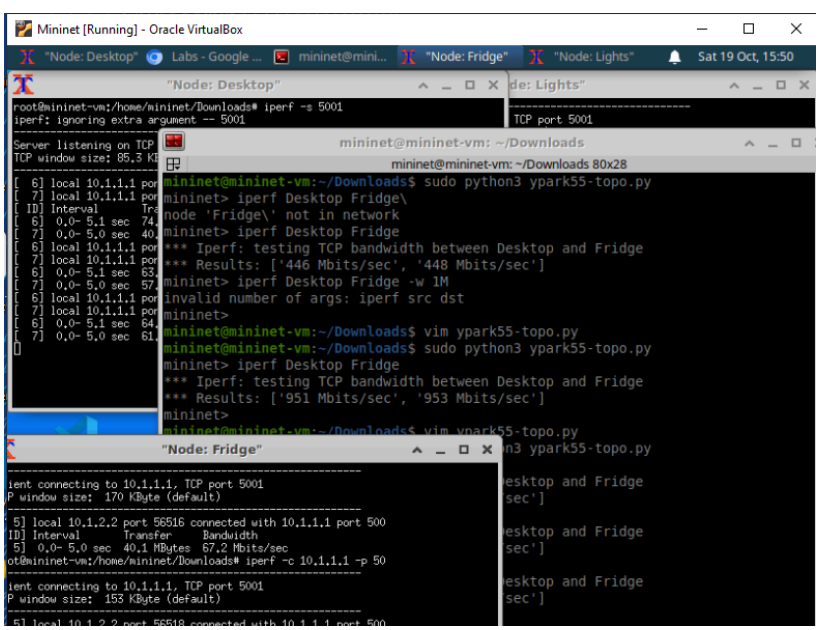
```
"Node: Fridge"
root@mininet-vm:/home/mininet/Downloads# iperf -c 10.1.1.1 -p 50
------------------------------------------------------------
Client connecting to 10.1.1.1, TCP port 5001
TCP window size:  170 KByte (default)
------------------------------------------------------------
[  5] local 10.1.2.2 port 56516 connected with 10.1.1.1 port 500
[ ID] Interval       Transfer     Bandwidth
[  5]  0.0- 5.0 sec  40.1 MBytes  67.2 Mbits/sec
root@mininet-vm:/home/mininet/Downloads# iperf -c 10.1.1.1 -p 50
------------------------------------------------------------
Client connecting to 10.1.1.1, TCP port 5001
TCP window size:  153 KByte (default)
------------------------------------------------------------
[  5] local 10.1.2.2 port 56518 connected with 10.1.1.1 port 500
[ ID] Interval       Transfer     Bandwidth
[  5]  0.0- 5.0 sec  63.4 MBytes   106 Mbits/sec
root@mininet-vm:/home/mininet/Downloads# iperf -c 10.1.1.1 -p 50
------------------------------------------------------------
Client connecting to 10.1.1.1, TCP port 5001
TCP window size:  484 KByte (default)
------------------------------------------------------------
[  5] local 10.1.2.2 port 56522 connected with 10.1.1.1 port 500
[ ID] Interval       Transfer     Bandwidth
[  5]  0.0- 5.0 sec  64.6 MBytes   108 Mbits/sec
root@mininet-vm:/home/mininet/Downloads#
```

Lights:



I couldn't fit all the screens in one screen shot because the VM screen couldn't be resized to a larger screen :((

Proof I did have 4 terminal windows:

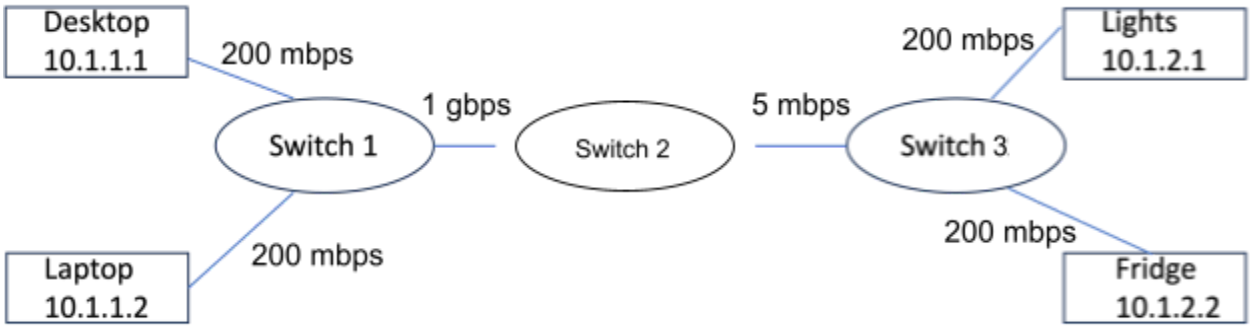


e. <u>The bottleneck link:</u> In this problem, we will observe the network with a bottleneck link. Add one more switch to your topology file, called Switch 3, which is placed in between switch 2 and the Fridge/Lights devices. In other words, the path from the Desktop to the Lights is 3 hops through Switch 1, Switch 2 and Switch 3. Set the bandwidth between Switch2-Switch3 to **5 Mbps**.

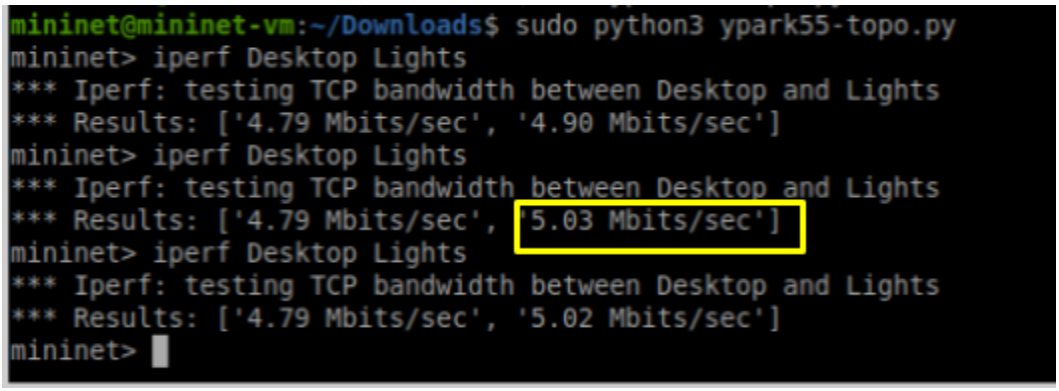
You can leave the rest of the links' bandwidth as it was configured in part d.

- Update topology drawing and label the bandwidth of all links on the path. Include this drawing with your submission.
- Now run **iperf** between Desktop and Lights 3 times and take the average throughput of the results. What is the average you observe? Take a **<u>screenshot</u>** and circle the lowest value observed.
- Discuss your results and how they have changed. Refer to the topology drawing and the iperf results. Are your results what you expected? Why or why not? Explain.

Topology:



Screenshot:



```
mininet@mininet-vm:~/Downloads$ sudo python3 ypark55-topo.py
mininet> iperf Desktop Lights
*** Iperf: testing TCP bandwidth between Desktop and Lights
*** Results: ['4.79 Mbits/sec', '4.90 Mbits/sec']
mininet> iperf Desktop Lights
*** Iperf: testing TCP bandwidth between Desktop and Lights
*** Results: ['4.79 Mbits/sec', '5.03 Mbits/sec']
mininet> iperf Desktop Lights
*** Iperf: testing TCP bandwidth between Desktop and Lights
*** Results: ['4.79 Mbits/sec', '5.02 Mbits/sec']
mininet>
```

The average is 4.887 Mbits/sec

The results were as expected. The bottleneck link between Switch 2 and Switch 3 has a bandwidth of 5 Mbps, which limits the maximum throughput between Desktop and Lights. As a result, the throughput measured by iperf aligns closely with the bottleneck bandwidth.

Footnotes:

[1] References to figures and sections are for the 8th edition of our text, *Computer Networks, A Top-down Approach, 8h ed., J.F. Kurose and K.W. Ross, Addison-Wesley/Pearson, 2020.*

[2] See https://www.howtogeek.com/304218/how-to-clear-your-history-in-any-browser/ for instructions on clearing your browser cache.

[3] *Hint:* ideally, you should see an If-Modified-Since header since you've just downloaded this page a few seconds ago. However, depending on the browser you're using, and the format of the server's earlier response to your initial GET, your browser may not include an If-Modified-Since even if the document has been downloaded and cached. The Chrome browser is pretty good at regularly using If-Modified-Since. But Safari and Firefox are much more finicky about when to use If-Modified-Since. Life isn't always as easy in practice as it is in theory!

Appendix:

Screenshot for Q3(b). I don't think it was needed but just in case

http

No.	Time	Source	Destination	Protocol	Length	Info
950	70.438291183	192.168.64.7	128.119.245.12	HTTP	516	GET /wireshark-labs/HTTP-wireshark-file4.html HTTP/1.1
954	70.516812105	128.119.245.12	192.168.64.7	HTTP	105	HTTP/1.1 200 OK (text/html)
961	72.000186177	192.168.64.7	128.119.245.12	HTTP	462	GET /pearson.png HTTP/1.1
965	72.078096126	128.119.245.12	192.168.64.7	HTTP	1165	HTTP/1.1 200 OK (PNG)
969	72.180330132	192.168.64.7	178.79.137.164	HTTP	441	GET /8E_cover_small.jpg HTTP/1.1
972	72.333375375	178.79.137.164	192.168.64.7	HTTP	237	HTTP/1.1 301 Moved Permanently
1409	76.702909926	192.168.64.7	128.119.245.12	HTTP	462	GET /favicon.ico HTTP/1.1
1410	76.778229042	128.119.245.12	192.168.64.7	HTTP	538	HTTP/1.1 404 Not Found (text/html)